**Method:**
For this problem I chose a greedy algorithm that keeps track of local imbalances to find the number of moves needed to evenly distribute the dresses across all machines

**Approach:**
The algorithm starts by checking whether the total number of dresses is divisible by the number of machines, because if not then you can return early as you can't evenly distribute non divisible amounts. Then it calculates the target number of dresses in each machine. Then it loops through each machine keeping track of the balance which represents the number of dresses that must pass through the machine.

**Reasoning:**
This algorithm works to determine the number of moves by calculating a local surplus, how many dresses must go out, and the cumulative imbalance which tracks how many dresses need to flow between the left and right. The total number of moves must satisfy both of these conditions and thus the max of the two constraints is the final result.

**Reflection:**
I used these test cases:

Input: machines = [1,0,5]
Output: 3

Input: machines = [0,3,0]
Output: 2

Input: machines = [0,2,0]
Output: -1

**Time Complexity:**
O(n)